

Shtyria Oleksii
Block course “Building GPT from scratch”
Prof. Elia Bruni
Group 29
Technical report
[Link to GitHub repository](#)
M. Sc. Cognitive Science
CS-MWP-AI Module

Introduction

This report concerns the UNIX stuff, bpe token learner, bpe text segmentor, simple n_gram next token generator, neural n_gram next token generator, simple transformer next token generator from [this repository](#):

A [cleaned version of the Shakespeare corpus](#) has been used for all of the above. The Shakespeare corpus has been split into the [training](#), [validation](#) and [test](#) parts.

UNIX commands

The results can be found in the jupyter notebook and [here](#).

BPE token learner

The purpose of a token learner is to come up with a list of subword tokens in an *a posteriori* data-driven manner. BPE or byte-pair encoding algorithm is one of the approaches to generate tokens from data in that way. [The implementation of the algorithm I opted for](#) operates upon a corpus representation in which each unique character is mapped onto the frequency of its occurrence in that corpus. Since all works of Shakespeare we have are written in English, the Shakespeare corpus is represented as a hash map from lowercase English alphabet characters to the number of times those characters occur in the case-folded corpus. English alphabet characters thus comprise the initial vocabulary or token list of the algorithm. Since there are 26 letters in the English alphabet, 1 special end-of-word token and the algorithm then runs a predefined number of iterations adding one new token to that list per iteration, the resultant vocabulary size is 27 plus the number of those iterations. The number of those iterations is usually denoted as ‘k’.

Each of those iterations the algorithm computes the frequency with which each pair of contiguous tokens occurs in the corpus and then appends the concatenation of the most frequent token pair to the vocabulary. It is not necessary to loop over the entire corpus to compute contiguous token pair frequencies. Instead, one can compute the frequency with which a given token pair occurs within a given word type and multiply that frequency with the frequency with which that word type occurs in a corpus. If one loops over all word types of the corpus and then

adds the products of frequency of token pair within word type with the frequency of word type in the corpus, the sum of those products is the number of times with which the token pair occurs in the corpus. After the most frequent pair of contiguous tokens is added to the vocabulary, the two tokens comprising that pair are replaced with their concatenation in the corpus. If the most frequent pair of tokens consists of ‘a’ and ‘b’, then words like ‘a-b-l-a-t-i-v-e’ are to be replaced with ‘ab-l-a-t-i-v-e’.

The process outlined in the previous paragraph should yield a subword token vocabulary with the size k plus 27. I generated [vocabularies of the size 427 to 2027](#) with the step size of 200. Token learning process is not a long one. Since the number of word types in a corpus is sublinear w. r. t. the number of word tokens in the same corpus according to the [Heap’s law](#), computing token pair frequency using word type frequencies has computational complexity which is *sublinear* in the corpus size as measured by the number of word tokens therein. Such an approach might thus scale well to corpora larger than the Shakespeare one.

BPE text segmentor

The goal of a text segmentor is to partition the corpus into tokens learned by the token learner. Token segmentor is built in a way similar to token learner. The corpus to be tokenized is represented as unique word types as well as their positions in the original corpus. Text segmentor loops over each vocabulary entry and then each word type in the corpus, splits that word type into individual letters and tries to merge those letters into the current vocabulary entry. If the current vocabulary entry is ‘ab’ the algorithm replaces all contiguous occurrences of ‘a’ and ‘b’ in word types like ‘a-b-l-a-t-i-v-e’ with ‘ab’ resulting in word types akin to ‘ab-l-a-t-i-v-e’. This is a greedy text segmentation: contiguous tokens are merged into vocabulary entries in the order in which those entries occur in the vocabulary i.e. in the order of their descending frequencies in the corpus on which the vocabulary learner has been run. If the vocabulary contains tokens ‘ab’ and ‘bla’ such that the former occurs more frequently in the training corpus than the latter, the charred word type ‘a-b-l-a-t-i-v-e’ is merged into ‘ab-l-a-t-i-v-e’ rather than ‘a-bla-t-i-v-e’. After all vocabulary entries have been iterated over, segmented word types are placed at the positions which they used to occupy in the original corpus to be segmented.

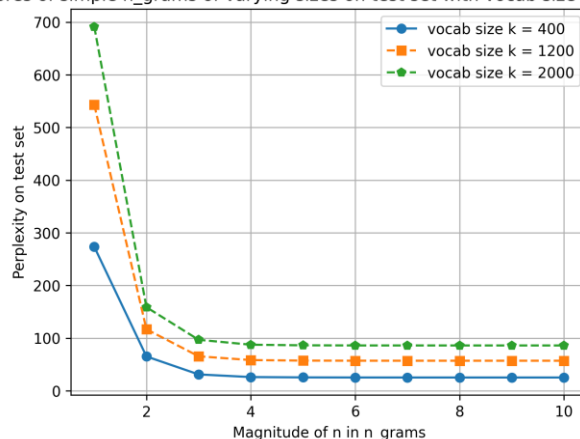
I [segmented the corpus using vocabularies of the size 427, 1227 and 2027](#). Since corpus segmentor also exploits the Heap’s law, *sublinear computational complexity* in the corpus size as measured by the number of tokens therein is to be expected.

Simple n _gram next token generator

As it was announced to be a viable option right from the start and since no hyperparameter optimization is expected from the “hardcore” track of the course anyway, I opted for the backoff version of the simple n _gram next token prediction algorithm. I hope that this can be compensated with the next token generation which has logarithmic computational complexity w. r. t. the number of n _grams of respective n .¹

Simple n _gram algorithm training corpus which is segmented with vocabularies of size 426, 1226 as well as 2026 from [here](#). It then generates n _grams of sizes 1 to 10 i.e. unigrams to 10-grams, gets their counts and also sorts them in the order of their descending frequencies. The algorithm then computes perplexity of the n _gram next token prediction on the test set via teacher forcing. To do so, the algorithm looks into the n _gram frequency counter to find the frequency with which sequence of length n occurs in the segmented training set. To get the perplexity of a given next token prediction, one has to divide the frequency mentioned in the previous sentence into the frequency of an $n-1$ _gram which contains all tokens of the respective n _gram but the last one. If one does not find n _gram in the list of n _grams, one backoffs to $n-1$ _grams. Then the frequency into which the $n-1$ _gram frequency is to be divided in order to find out perplexity is the frequency of the respective $n-2$ _gram. When the algorithm backoffs to a smaller n _gram, a constant value of 0,75 is deducted from the frequency of the smaller n _gram prior to division by the frequency of $n-1$ _gram to penalize the frequency scores of n _gram models which backoff more often.

Perplexity scores of simple n _grams of varying sizes on test set with vocab size $k = (400, 1200, 2000)$



¹ I did not have time to replace backoff with interpolation precisely because I spent a couple of days trying to come up with simple n _gram next token generation which is sublinear in the number of respective n _grams.

Figure 1. Perplexity scores of different n _gram model in relation to n and k from [here](#).

As for the next token generation method, it is logarithmic in the number of n _grams. The algorithm does not loop through the list of all n _grams to find the one which contains $n-1$ last tokens of a sequence to be appended next tokens to and at the same time has the largest frequency. Instead, the algorithm runs two binary searches over the list of lexicographically sorted n _grams to identify positions of the first and the last n _gram therein such that their prefix of the size $n-1$ matches $n-1$ last tokens of the sequence to be appended. Once positions of those first and last n _grams with correct prefixes have been identified, one only needs to loop through n _grams whose indices in the sorted n _gram list fall within the interval delineated by those first and last positions. The computational complexity of n _gram generation falls from the number of n _grams to the double logarithm of number of n _grams times the number of n _grams whose $n-1$ prefix matches last $n-1$ tokens of the sequence to be appended. The last token of the n _gram from that interval would be the next token prediction of the algorithm. Counterintuitively in the context of figure 1, n _gram models with $n=10$ produce more meaningful text both syntactically and semantically as vocabulary size increases. Here is the text generated by respective models from [here](#):

- vocab size $k = 400$, generated sequence is: “true king is dfor and rolonius i did enact julius caesar i was killed i the capitol brutus killed me hamlet it was a brute part of him to kill so capital a calf there be the players ready rosenkrantz ay my lord they stay upon your patience queen gertrude come hither my dear hamlet sit by me hamlet no good mother here s metal more attractive lord polonius to king claudius o ho do you mark that hamlet lady shall i lie in your lap lying down at ophelia s feet ophelia no my lord hamlet i mean my head upon your lap ophelia ay my lord hamlet do you think i meant country matters op”
- vocab size $k = 1200$, generated sequence is: “true king is lord you chard hath ven me by claudio he received them of him that brought them king claudius laertes you shall hear them leave us exit messenger reads high and mighty you shall know i am set naked on your kingdom to morrow shall i beg leave to see your kingly eyes when i shall first asking your pardon thereunto recount the occasion of my sudden and more strange return hamlet what should this mean are all the rest come back or is it some abuse and no such thing laertes know you the hand king claudius tis hamlets character naked and in a postscript here he says alone can you advise me laertes i m lost in it my lord but let him come it warms the very sickness in my heart that i shall live and tell him to his teeth thus didest thou king claudius if”
- vocab size $k = 2000$, generated sequence is: “true king is lord you cheantonio you too at each ear a hearer that great baby you see there is not yet out of his swaddling clouts rosenkrantz happily he s the second time come to them for they say an old man is twice a child hamlet i will prophesy he comes to tell me of the players mark it you say right sir o monday morning twas so indeed lord polonius my lord i have news to tell you hamlet my lord i have news to tell you when roscius was an actor in rome lord polonius the actors are

come hither my lord hamlet buz buz lord polonius upon mine honour hamlet then came each actor on his ass
lord polonius the best actors in the world either for tragedy comedy history pastoral pastoral comical
historical pastoral scene individable or poem unlimited sene”

Neural bigram next token generator

Albeit the name of the model might suggest otherwise, neural bigrams hardly have anything to do with a neural network like a multilayer perceptron. A neural bigram model is nothing else than a square array of parameters. The size of that array is equal to vocabulary length times vocabulary length. Column j of row i of that table contain the relative likelihood that j -th entry of vocabulary follows i -th entry of vocabulary. When normalized, those likelihoods or logits represent the conditional probability distribution of token j given token i . To get the loss of a model given input token i and some target label, one has to normalize logits in the i -th row of the model parameter array and then compute cross entropy loss between the target label and the normalized logit of the target label. I opted to optimize that loss with stochastic gradient descent as well as RMSprop. SGD is the simplest optimizer to implement; as such it is a default benchmark to be surpassed by more sophisticated optimizers. RMSprop is the most advanced optimizer I can argue not too implausibly to be able to wrap my mind around; Adam has so far escaped the grip of my intuition.

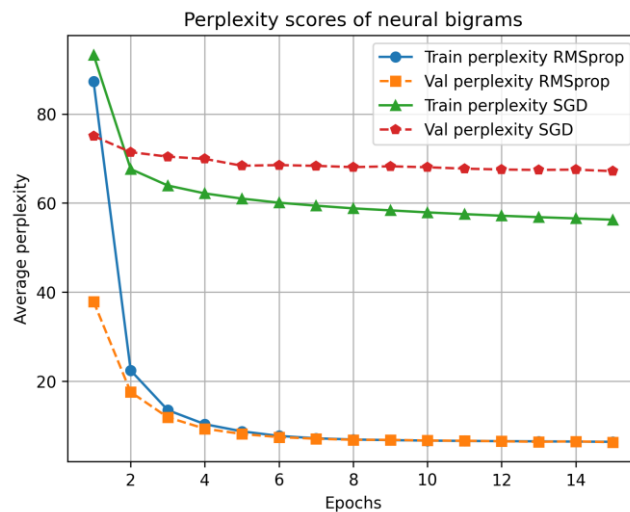


Figure 2. Perplexity scores of neural bigram model optimized via stochastic gradient descent and RMSprop from [here](#)

Nano GPT

A small version of GPT with causal self-attention has been implemented. I also used RMSprop with the same hyperparameters rho/alpha and epsilon as with neural bigrams to optimize cross

entropy loss. The difference is that L2 weight decay regularization is used on top of RMSprop to minimize the cross-entropy loss of GPT. Since neural bigrams have been implemented with numpy whereas GPT with pytorch, I had to tune hyperparameters of the latter. Numerous logs [here](#), [here](#) and [here](#) show that large learning rate of $5e-3$ is useless, that embedding dimension of less than 64 is insufficient, that increasing the number of hidden layers to 8 or 16 has limited benefits compared to 4 hidden layers. Vocab size K of 1200 and 2000 makes little sense. After grid search for 5 epochs the most successful model has been trained for a number of further epochs. Selected results are below.

Generated text from [here](#): “the tragel of a liberty cursed in the flowing of blood of the soul of blood and the watched in the blood of beauty s blos”.

Generated text from [here](#): “the tragel of a cold a great the bones and the soldius caesar i will you to breat the blood of the wealue of my dends the sleman and a”.

Generated text from [here](#): “the tragemia ct and i have not be daflt and i ll be a col and i will not a bles to you have belie i have better i will be a sinded to the couded to the ss and bressiefook and to the counturse of the bled and i have you shall be the brouse of my bldabring to the bray to the tragenteel of my lord pt of the bill of the c d the ss to the ct and the bressoooool and the ss of the ss and i corn d and bill be soooil and the ss of the ct of egemes of the brabil and i have bldased and ss the sed and bldrain the ss of the ct of this biturnattried of epured and blt of the sed and the brain his blt of emes and the ct to the ct of the ct and the ct of the bldabilutriving and ss the col and the ct bil and bill the sed bil d and the soilding to the coill of emes of his trieding of biluldrail of the cooooooooooot and the sed and sed and sed and ss of the bil d ct and the sed to the bil of the blt brailt and the soill of the bitriegried and the bice and the seast of the ct and the seace of el of el and sed and bitrain his bitried of the coil and brain the bluldaulult of the bice of the cans and bil and the brabi”.

Here are some loss plots:

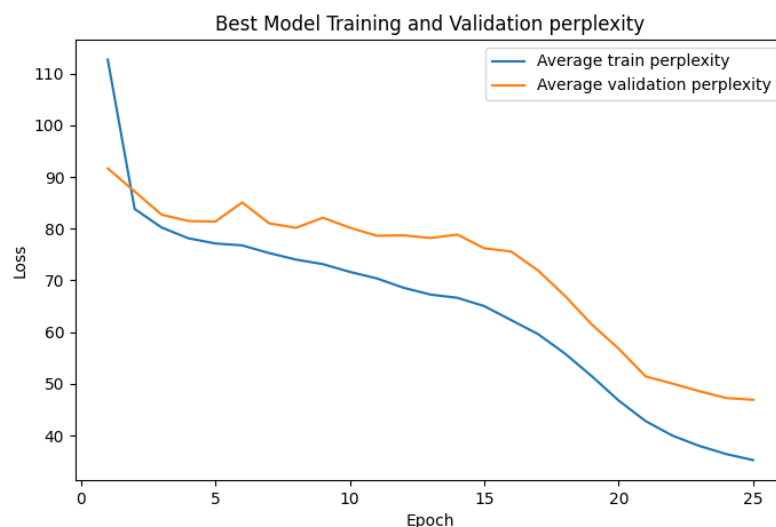


Figure 3. Loss histories of the model with hyperparams $K=400$, $lr=0.001$, $layers=4$, $embed_dim=128$, $heads=4$, $max_context=128$ from [here](#).

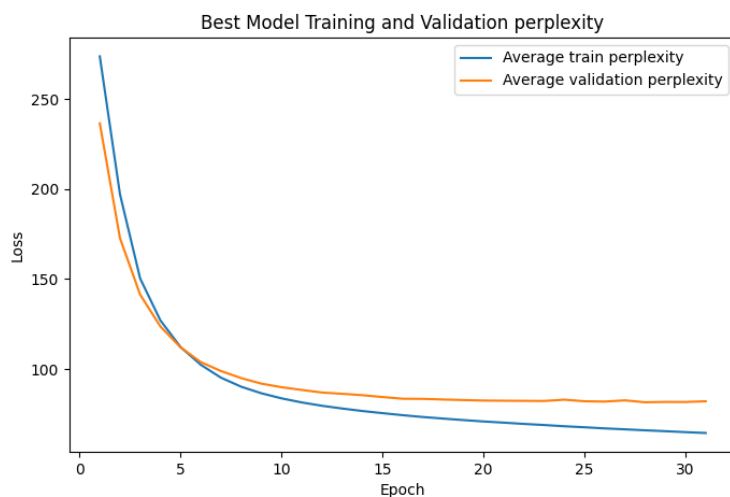


Figure 4. Loss histories of the model with hyperparams $K=400$, $lr=0.0001$, $layers=4$, $embed_dim=64$, $heads=4$, $max_context=128$ from [here](#).